
BUS TIMETABLE AND TICKETING SYSTEM(CATANBUS)

Group Name: Catan

Student Name	Student Number
Anastasia Bholee	M01015673
Ivan Pavlovskii	M00957512
Leo D'Hotman de Villiers	M00694663
Terry Leung Chew	M00983500
Ore Orekoya	M00982779

Professor's Name: Gina Soopramanien

Course: 2025-26 CST2550 Software Engineering Management and Development

Submission Date : 14/04/2026

Word Count : 2772

TOC for Report

Table of Contents

TOC for Report	1
Table for Figures	1
1. Introduction	2
1.1. Project Overview	2
1.2. Report Structure	2
2. Design	2
2.1 System Overview / Architecture	2
2.2 Data Structure Selection & Justification	3
2.2.1 Time Complexity Analysis	4
2.2.2 Space Complexity	4
2.2.3 Suitability of the problem	4
2.3 Algorithm Design (Key Functionality & Using Pseudocode Only)	5
2.3.1 Insert Operation	5
2.3.2 Load Operation	5
2.3.3 Search Operation	5
2.3.3 Delete Operation	5
2.4 Database Design	6
3. Testing	7
3.1 Testing Approach	7
3.2 Test Cases Table	8
4. Conclusion	9
4.1 Summary Of Work	9
4.2 Limitations	9
4.3 Critical Reflection	9

Table for Figures

Figure 1 : ERD diagram	6
------------------------------	---

Introduction Section

1. Introduction

1.1 Project Overview

CatanBus is a Desktop Bus Timetable and Ticketing System coded in C# (.NET 8), WinForms, and SQL Server. The project was developed to solve the problem of traditional paper-based bus management methods. In many small-scale transport services, scheduling and seat tracking are often handled manually, which results in frequent errors such as overbooking, lost records, difficulty in managing schedule cancellations. CatanBus solves these issues by providing a digital platform where everything is linked and synchronized with every transaction and also one can order their bus tickets from the comfort of their homes. The system enables users to register, log in, search for bus routes, purchase and view their tickets.

The system provides services for three types of users:

- **Moderator** : Manages users and admins
- **Administrators** : Manages routes, buses and can add, cancel or delete bus schedules.
- **Users** : Searches for available trips and book tickets.

The application follows a layer architecture (Forms and Services, Models and Database layers). SQL Server is used for persistent data storage while ADO.NET communicates between the application and the database. Unit testing through MS Test lets reliability of service layer is tested properly.

1.2. Report Structure:

This report is structured into four main sections including the introduction. Section 2 (Design) covers the architectural choices, data structures, algorithmic and complexity analyses. Section 3 (Testing) details the verification approach and test cases. Section 4 (Conclusion) provides all the implementations and summary of the project, limitations, and critical reflection on the development process.

Design Section

2. Design

2.1 System Overview / Architecture

The CatanBus system is developed using a 3-Tier Layered Architecture.

► **Presentation Layer (UI)**: Implemented using C# Windows Forms, this layer manages all user interactions. It includes:

- **Login Form** : Allows users to login using their email and password
- **Register Form** : Allows new users to create an account using password validation rules including a capital letter, number, and symbol
- **User Dashboard** : Available trips, trip filtering (by route and date) and ticket booking and viewing are some of the features available to users.

- **Admin Dashboard** : Admins can add buses, make trips, view bookings made and cancel trips. Cancelling a trip automatically updates status of ticket and processes refunds. They can also delete the trips which have no bookings.
- **Moderator Dashboard** : Moderator can search for users and upgrade a normal user to Admin level access or vice-versa.
- **Booking and Payment** : User can choose number of seats to book and confirm his bookings by payment process.

► **Business Logic Layer (Services)**: This middle layer acts as the "brain" of the application. It contains the service classes (e.g., TicketService.cs, TripService.cs) that process business rules, such as checking seat availability before allowing a booking or calculating total prices.

► **Data Access Layer (Persistence)**: This layer handles communication with the SQL Server database via ADO.NET. It utilizes the DatabaseService.cs class to manage connections and execute the CRUD (Create, Read, Update, Delete) operations required by the services.

2.2 Data Structure Selection & Justification

The application follows the layered architecture to separate the responsibility and achieve the improved maintainability.

► **Forms Layer** : This layer houses the graphical user interface which has been implemented by using WinForms. Some examples of request path variables are LoginForm, RegisterForm, UserDashboardForm, AdminDashboardForm, BookingForm, PaymentForm.

► **Service Layer and Generic Lists** : This layer contains the core business logic. Services interface with the database through the DatabaseService class. Primary services include AuthenticationService, BusService, TripService, TicketService, PaymentService, and UserService. The List<T> (Generic List) was used as the primary data structure throughout the project to manage collections of trips, users, and buses. Arrays were not used because they have a fixed size (static space allocation). Since the number of bus trips in the database grows daily, a fixed array would lead to "Index Out of Range" errors or wasted memory. In contrast, a List<T> resizes dynamically.

- **Time Complexity**: Accessing an element by index in a List<T> is $O(1)$, while searching for a specific trip or user via a linear search results in $O(n)$ complexity. This provides the necessary performance for handling the current data volume efficiently.

► **Models Layer**: Models represent database entities such as User, Bus, Trip, Ticket and Payment. Instead of passing loose strings, a Trip object ensures that Price is always a decimal and SeatsAvailable is always an integer. It prevents data corruption or errors during the booking process.

► **Static Session Object (Session Management)**: A Static Property was implemented in Session.cs (public static User currentUser), Instead of creating a new user object on every form (which wastes RAM), a single static instance is kept in memory of the application, providing $O(1)$ access time to user permissions (Admin/ Moderator)

► **Data Layer and SQL Tables** : The Data Layer manages connectivity to the SQL Server using ADO.NET. The data is organized into a relational schema consisting of five tables: Users, Bus, Trip, Ticket, and Payment.

- **Primary Keys:** Each table implements a Primary Key (e.g., UserID, TripID) using the INT IDENTITY property. This ensures Entity Integrity, providing a unique identifier for every record and allowing for fast indexed searches.
- **Foreign Keys:** Relationships are established through Foreign Keys (e.g., BusID in the Trip table, TripID in the Ticket table). The purpose of these relationships is to maintain Referential Integrity; for example, the system ensures that a ticket cannot be booked for a trip that does not exist in the database, and a bus cannot be deleted if it is currently assigned to an active route.

2.2.1. Time Complexity Analysis

The system's performance is determined by how efficiently it handles data within the List<T> structures and the SQL database. Key operations in the system have the following time complexities:

- **Search Trips** – $O(n)$: This operation is performed in the UserDashboardForm.cs using LINQ to filter a List<Trip>. Because the system must check every individual trip in the list to see if it matches the user's Route Constraints (Origin and Destination), the time taken grows linearly with the number of trips (n).
- **Load Trips** – $O(n)$: Implemented in TripService.GetAllTrips(), this operation involves iterating through a SqlDataReader to populate a List<Trip>. Each row returned by the SQL query must be mapped to a C# object and added to the list, requiring one operation per record (n).
- **User Search** – $O(n)$: In UserService.cs, searching for a user involves a linear scan of the Users table or the resulting List<User>. Even with SQL indexing, the process of iterating through the collection to display matching keywords in the ModeratorDashboardForm follows a linear growth pattern.
- **Ticket Booking** – $O(1)$: This is a direct INSERT operation in the TicketService.cs. Since the system does not need to search the table to add a new record, but rather appends it to the end of the data store using a Primary Key identity, the operation takes a constant amount of time regardless of the database size.
- **Trip Cancellation** – $O(n)$: This is an operation in TripService.CancelTrip(). It is $O(n)$ because the system must identify and update all n tickets associated with that specific TripID. In the backend, the SQL engine performs a scan to find matching records to change their status to 'Cancelled', making the complexity proportional to the number of bookings for that trip.

Although database operations dominate the processing time, the use of structured queries ensures efficient performance for typical system usage.

2.2.2. Space Complexity:

Space complexity is $O(n)$, where n is the number of records retrieved. The system is designed to be memory-efficient by only loading active trips into the RAM, rather than the entire historical database.

2.2.3. Suitability of the problem

The combination of relational SQL tables for persistent storage and List<T> for in-memory processing is ideal for this system. This architecture ensures safety and reliability with no risk of duplicate bookings through database-level constraints. Additionally, dynamic lists facilitate rapid UI updates, achieving a balance between transactional integrity and the high-speed performance required for real-time ticketing.

2.3 Algorithm Design (Key Functionality & Using Pseudocode Only)

2.3.1. Insert Operation

BUSSERVICE.CS (Method: AddBus)-handles the inventory by inserting new Bus records into the database with their full seating capacity.

```
FUNCTION AddBus(busNumber, seatsCapacity)
    OPEN database connection

    // Set Capacity and Available seats to the
    input value
    SET query = "INSERT INTO Bus (BusNumber,
    SeatsCapacity, SeatsAvailable) VALUES
    (busNumber, seatsCapacity, seatsCapacity)"

    INITIALISE command with parameters
    EXECUTE non-query command
    CLOSE connection
END FUNCTION
```

2.3.2. Load Operation

TRIPSERVICE.CS (Method: GetAllTrips)-Fetches all scheduled routes by merging trip and bus tables and filters for 'Active' status only.

```
FUNCTION GetAllTrips()
    INITIALISE empty list for trips
    OPEN database connection

    // Note: Code joins Bus table to get BusNumber and
    SeatsAvailable
    SET query = "SELECT Trip and Bus details WHERE
    Trip.Status is Active"
    INITIALISE data reader

    WHILE record found in data reader
        INITIALISE trip object
        MAP database columns (Price, Departure,
        BusNumber, etc.) INTO trip object properties
        ADD trip object TO trips list
    END WHILE
    CLOSE connection
    RETURN trips list
```

2.3.3. Search Operation

USERDASHBOARDFORM.CS (Method: btnSearch_Click)- Processes user input for destination and passenger count to filter the list of available trips.

```
FUNCTION SearchAndFilter(Origin, Destination,
Passengers, TravelDate)
    INITIALISE allTrips = CALL GetAllTrips()

    IF Origin is selected THEN
        FILTER allTrips where FromLocation
        matches Origin
    IF Destination is selected THEN
        FILTER allTrips where ToLocation
        matches Destination
    FILTER allTrips where Departure Date
    matches TravelDate

    IF Passengers > 0 THEN
```

2.3.4. Delete Operation

TRIPSERVICE.CS (Method: DeleteTrip)-Removes a specific trip from the database using its unique identifier.

```
FUNCTION DeleteTrip(tripId)
    OPEN database connection

    SET query = "DELETE FROM Trip WHERE TripID =
    tripId"
    INITIALISE command with tripId parameter

    EXECUTE non-query command

    CLOSE connection
    RETURN
```

FILTER allTrips where SeatsAvailable >= Passengers

DISPLAY the final filtered list in the UI grid
END FUNCTION

2.4 Database Design

SQL was used due to its reliability, organization, and querying capabilities, this programming language makes it easy to create, update and work with a database.

This is why the database engine used in the system is SQL server. The database called CatanBusDB has all application data such as users, buses, trips, tickets and payments.

- **Users Table**-Contains user accounts such as moderators and admins.
- **Bus Table**-Stores information about bus such as capacity and free seats.
- **Trip Table**-Stores available routes in terms of departure/arrival and price.
- **Ticket Table**-Stores ticket booking information for the users.
- **Payment Table**-Stores payment records, status and payment date.

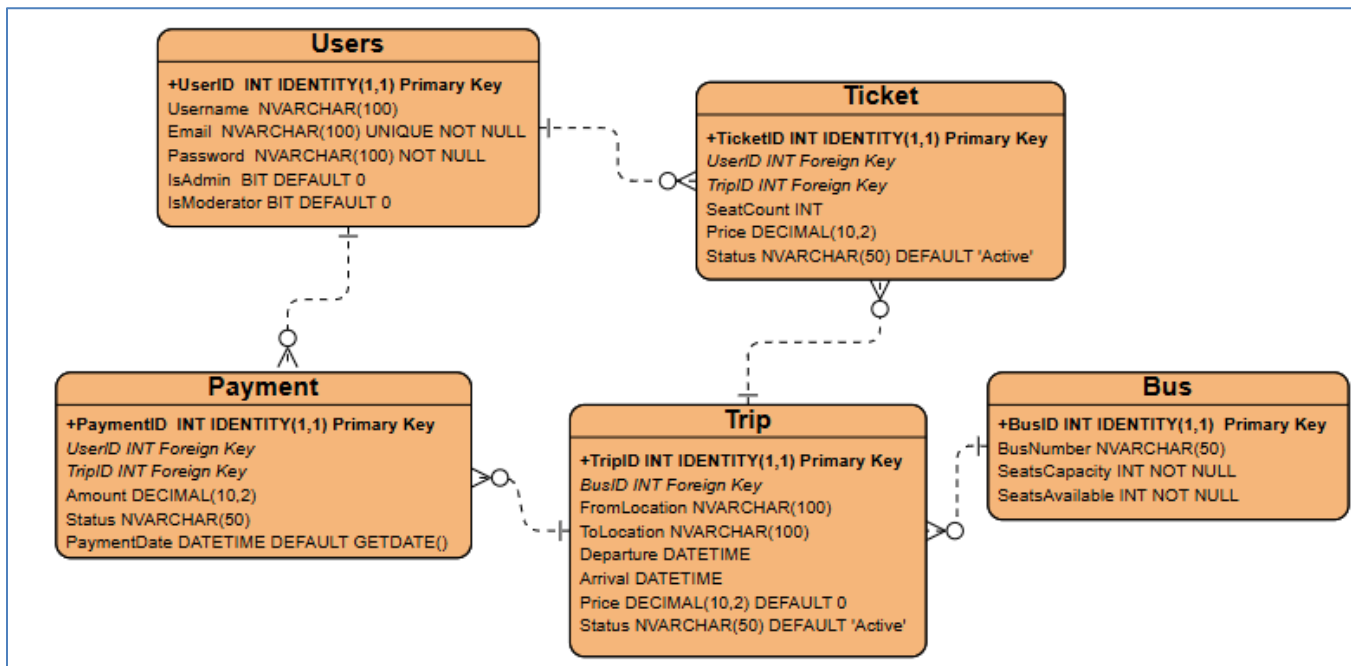


Figure 3 : ERD diagram

The different cardinality types in our database are :

- Bus => Trip : One to Many (One bus has many trips)
- Trip => Payment : One to Many (One trip has many payments)
- Trip => Ticket : One to Many (One trip has many tickets)
- Users => Ticket : One to Many (One user has many tickets)
- Users => Payment : One to Many (One user has many payments)

Database Interaction (Read/Write Operations)

The application interacts with the SQL Server database through the **ADO.NET** framework within the DatabaseService and specific Service classes:

- **Create:** Uses INSERT commands (e.g., in TicketService.BookTicket) to add new records.
- **Read:** Uses SqlDataReader and SqlDataAdapter (e.g., in TripService.GetAllTrips) to fetch data and map it to class instances or DataTable objects.
- **Update:** Uses UPDATE commands (e.g., in UserService.MakeAdmin) to modify existing records.
- **Delete:** Uses DELETE commands (e.g., in TripService.DeleteTrip) to remove data while maintaining referential integrity.

Considering Normalization, for the First Normal Form, in CatanBusDB all attributes of all relations only have one value; then, for the Second Normal Form, in our database all relations cannot exist without their ID which is the primary key in all relations; finally, for the Third Normal Form, in our relational database there is no transitive dependencies, all attributes of all relations depend on their primary key.

Testing Section

3. Testing

3.1 Testing Approach

The system was tested using **integration testing** as it interacts with a SQL Server database. Testing was carried out using Microsoft Visual Studio with the MSTest framework, allowing automated execution of test cases. The following modules were tested: User Authentication (Registration and Login), Ticket Booking System, User Management, Trip Management, and Payment Processing

These modules were selected as they represent the core functionality of the Bus Ticketing System.

The testing was conducted in the following environment:

- Development Tool: Visual Studio
- Programming Language: C# (.NET Framework)
- Database: SQL Server
- Testing Framework: MSTest

The system was connected to a live database, allowing real-time validation of data operations such as inserts, updates, and retrievals.

The authentication module was tested for valid and invalid login scenarios and user registration.

The ticket booking module ensured users could book and retrieve tickets successfully.

User management testing confirmed admin functions such as retrieving, searching, and updating users.

Trip management verified that trips could be created, retrieved, updated, and deleted.

Payment processing ensured transactions were correctly recorded and tracked.

3.2 Test Cases Table

Test Case	Description	Expected Output	Result
Register Valid User	User enters valid details	User successfully registered	Pass
Register Duplicate User	Same email used twice	Registration fails	Pass
Login Valid Credentials	Correct email and password	User logged in	Pass
Login Invalid Password	Incorrect password	Login fails	Pass
Login Non-Existing User	Email not in system	Login fails	Pass
Book Ticket (Valid)	User selects valid trip and seat count	Ticket successfully booked	Pass
Book Ticket (Max Seats)	User books available seats	Booking successful	Pass
Retrieve User Tickets	User views their tickets	List returned	Pass
Get All Tickets	Admin retrieves all tickets	Data displayed	Pass
Multiple Bookings	User books multiple tickets	All bookings recorded	Pass
Get All Users	Retrieve all registered users	List of users returned	Pass
Search Users (Valid)	Search using existing username/email	Matching users returned	Pass
Search Users (No Match)	Search with non-existing keyword	Empty list returned	Pass
Make Admin	Assign admin role to user	User updated successfully	Pass
Get All Trips	Retrieve active trips	List returned	Pass
Add Trip	Add new trip to system	Trip successfully added	Pass
Cancel Trip	Cancel an existing trip	Trip status updated	Pass
Trips With Bookings	Retrieve trips with booking data	Data returned	Pass
Delete Trip	Remove trip from system	Trip deleted	Pass
Process Payment	Valid payment details entered	Payment recorded	Pass
Multiple Payments	Multiple transactions processed	All recorded correctly	Pass

Payment Status	Check payment status	Status = "Paid"	Pass
-------------------	-------------------------	-----------------	------

The testing confirmed that the system is **functional, stable, and meets its requirements**. The use of MSTest ensured efficient and reliable validation of all core features.

Conclusion Section

4. Conclusion

4.1. Summary Of Work

The CatanBus Bus Timetable and Ticketing System is a successful implementation of a complete ticket reservation platform using C#. Net and WinForm and also with SQL server.

The data structure used was List which resizes dynamically, compared to Arrays. This is a must need feature due to the bus trips added daily.

The functionality provided includes user authentication, route management, ticket booking, payment etc. and administrative controls; The layered architecture helps in better maintainability.

4.2. Limitations

Some limitations we had were, for example, a technical issue due to a "Build Error MSB3821 (Mark of the Web)", which prevented the processing of a file; this issue was resolved.

Then, considering our data structure, if we had a lot of data, the performance could decrease if the list exceeds its current capacity, and by doing so, doubled in size. Finally, one of the design challenges of our project was to create three dashboards for three different roles, which means designing three different user paths.

4.3. Critical Reflection

What could be done differently next time includes the improvement of security functions such as the encryption of passwords. But also, the usage of the async / await keyword for all I/O operations to improve scalability. And finally, regarding the design, the different dashboards would be improves in term of user experience.